Week 5 & 6: React.js Architecture

Glaxit MERN Stack Internship Program

Transitioning from Vanilla JavaScript to component-driven UI development. These weeks focus on building robust, scalable frontends using React, functional components, and modern hooks.

Week 5: React Fundamentals & State

Day 1: Component Architecture & JSX

- **Study:** Setting up a React environment (Vite vs CRA), JSX syntax, rendering elements, and the difference between Functional and Class components.
- **Task:** Bootstrap a new React application using Vite. Create a static landing page layout broken down into distinct components (`<Navbar />`, `<Hero />`, `<Footer />`). Pass basic string data into these components using `props`.

Day 2: State Management & Event Handling

- **Study:** Introduction to the `useState` hook. Handling user events (onClick, onChange) in React.
- **Task:** Build an interactive "FAQ Accordion" component. Map over an array of question/answer objects and use local state to track which accordion item is currently expanded, ensuring only one item can be open at a time.

Day 3: The React Lifecycle & useEffect

- **Study:** The component lifecycle (mount, update, unmount). Managing side effects and dependency arrays using the `useEffect` hook.
- **Task:** Create a component that fetches mock data (e.g., placeholder posts) from an API immediately upon mounting. Display a loading spinner while the fetch is pending, and handle potential API errors gracefully in the UI.

Day 4: Routing & Navigation

- **Study:** Client-side routing with `react-router-dom`. Setting up `<BrowserRouter>`, `<Routes>`, `<Route>`, and navigation using `<Link>` or `useNavigate`.
- **Task:** Implement multi-page navigation in your React app. Create separate views for Home, About, and a dynamic 404 Not Found page.

Day 5: Dynamic Routing & URL Parameters

- **Task:** Extend the Day 4 routing setup by adding a "Products" page that displays a grid of items. Clicking an item should route the user to a specific detail page (e.g., `/products/:id`) using URL parameters to fetch and display that specific item's details.

Week 6: Advanced React & Portfolio Capstone

Day 1: Form Handling & Controlled Inputs

- **Study:** Controlled vs. Uncontrolled components. Managing complex form state with a single state object.
- **Task:** Build a comprehensive "Contact Us" form component. Ensure all inputs (text, email, textarea) are strictly controlled by React state. Add real-time validation (e.g., disabling the submit button until the email format is correct).

Day 2: Global State & Context API

- **Study:** Avoiding "prop drilling" using the React Context API and the `useContext` hook.
- **Task:** Implement a global "Theme Context". Wrap your application in a provider that stores the current theme (light or dark mode). Add a toggle button in your Navbar that flips this global state and instantly updates the CSS variables across the entire application.

Day 3: Performance Hooks & Refs

- **Study:** Memoization and preventing unnecessary re-renders using `useMemo` and `useCallback`. Direct DOM manipulation using `useRef`.
- **Task:** Create a custom video player component. Use `useRef` to access the underlying HTML5 `<video>` element to implement custom play, pause, and mute buttons that bypass standard browser controls.

Day 4 & 5: Capstone Project Deployment

THE REACT PORTFOLIO WEBSITE

Consolidate your knowledge from the past 6 weeks to build and deploy a professional, interactive developer portfolio. This application will serve as your digital resume.

Core Requirements:

- **Component Architecture:** Break the UI down intelligently. Use a central layout component that persists the Navbar and Footer across all routes.
- **Dynamic Project Gallery:** Create a mock JSON array containing your best projects (include your tech stacks like Node.js, MERN, AI frameworks, or VR environments). Use array mapping to generate reusable `<ProjectCard />` components.
- **Routing:** Utilize `react-router-dom` for seamless client-side transitions between Home, About, Projects, and Contact pages.
- **Global State:** Integrate the Dark/Light mode toggle built on Week 6, Day 2 via the Context API.
- **Form Interaction:** Implement a controlled Contact Form. (Bonus: hook it up to a service like EmailJS to actually receive the messages).
- **Optimization:** Ensure your project assets (images, fonts) are optimized and your component tree is clean.
- **Deployment:** Push the final build to Vercel, Netlify, or GitHub Pages. The repository must include a well-structured README detailing your design decisions.